receiving data in JSON format, just like an SPA. Then it modifies the application's UI depending on the data it receives.

One final use case for an HTTP API is where your application is designed to be partially or solely consumed by other backend services. Imagine that you've built a web application to send emails. By creating an HTTP API, you can allow other application developers to use your email service by sending you an email address and a message. Virtually all languages and platforms have access to an HTTP library they could use to access your service from code.

That's all there is to an HTTP API: it exposes endpoints (URLs) that client applications can send requests to and retrieve data from. These endpoints are used to power the behavior of the client apps, as well as to provide all the data the client apps need to display the correct interface to a user.

NOTE You have even more options when it comes to creating APIs in ASP.NET Core. You can create remote procedure call APIs using gRPC, for example, or provide an alternative style of HTTP API using the GraphQL standard. I don't cover those technologies in this book, but you can read about gRPC at https://docs.microsoft.com/aspnet/core/grpc and find out about GraphQL in *Building Web APIs with ASP.NET Core*, by Valerio De Sanctis (Manning, 2023).

Whether you need or want to create an HTTP API for your ASP.NET Core application depends on the type of application you want to build. Perhaps you're familiar with client-side frameworks, or maybe you need to develop a mobile application, or you already have an SPA build pipeline configured. In each case, you'll most likely want to add HTTP APIs for the client apps to access your application.